

Grille de décision

Réunion de convergence d'architecture — un point à la fois

Ce document liste, point par point, les choix techniques sur lesquels des propositions divergent (Spring Boot / FastAPI / Flask, Streamlit / React, Chroma / Qdrant, cache, LLM local / cloud...). Pour chaque point : les options en présence, le critère qui doit trancher, et la réponse selon le scénario retenu au point 0. **Objectif de la réunion : passer ces points un par un et acter une décision pour chacun.**

0. Question préalable — quel est le produit, et pour qui ?

Avant tout choix technique : ce projet est-il un ensemble de stages individuels de 8 semaines, chacun livrant un prototype défendable seul — ou un produit unique construit à plusieurs, sur un horizon plus long ? **La réponse conditionne presque toutes les décisions suivantes** ; c'est pour cela qu'elle doit être tranchée en premier, explicitement, et non déduite après coup des choix technique.

Scénario A — Stages individuels, 8 semaines

Chaque personne livre un prototype qu'elle comprend et défend seule. Priorité : compréhension, reproductibilité, périmètre tenable en solo.

Scénario B — Produit commun, équipe, horizon > 8 semaines

Plusieurs personnes maintiennent le même code dans la durée. Priorité : séparation des responsabilités, conventions partagées, scalabilité.

1. Souveraineté du LLM — local vs cloud

Question à trancher : Le LLM tourne-t-il en local (Ollama, Qwen3/Mistral) ou via une API cloud (Gemini, GPT) ?

Options sur la table :

- **Local** — Ollama + Qwen3 8B / Mistral 7B, quantifié, aucune donnée ne quitte le poste.
- **Cloud** — API Gemini / GPT-4, latence et qualité potentiellement meilleures, mais la question du citoyen part vers un tiers externe.

Ce qui doit décider : Ce n'est pas un arbitrage d'ingénierie comme les autres — c'est une question d'identité produit. Si « souverain » est un engagement réel du projet (le nom même le porte), le cloud est exclu par principe, quels que soient la taille de l'équipe ou le calendrier.

Ce point doit être tranché en premier, avant tout le reste. Il détermine la faisabilité même du mot « souverain » dans le nom du produit, et conditionne la contrainte de capacité qui influence ensuite chaque autre choix (un LLM local impose des limites qu'une API cloud n'impose pas).

2. Langage et framework backend métier

Question à trancher : Java/Spring Boot, Python/FastAPI, Python/Flask, ou aucun framework (script direct) ?

Options sur la table :

- **Spring Boot** (Java) — typé, mature, écosystème entreprise.
- **FastAPI** (Python) — async natif, OpenAPI auto-généré, même écosystème que le RAG.
- **Flask** (Python) — minimal, flexible, faible courbe d'apprentissage.
- **Aucun framework** — boucle RAG codée à la main.

Ce qui doit décider : Le langage du backend métier doit être celui de l'équipe qui le maintient réellement. Introduire Java quand le cœur RAG est en Python force soit un pont HTTP entre deux runtimes (latence, logique dupliquée), soit deux équipes séparées par compétence linguistique.

Si stage(s) solo, 8 semaines

Python partout, pas de Java. Une personne seule n'a pas le temps d'apprendre et de maintenir deux écosystèmes en 8 semaines. Framework minimal, voire absent, pour comprendre et défendre chaque étape en soutenance.

Si produit d'équipe, long terme

FastAPI est plus cohérent que Spring Boot si le cœur RAG reste en Python : même langage, pas de pont HTTP inutile, typage via Pydantic. Spring Boot ne se justifie que si l'équipe a une expertise Java préexistante forte, ou une contrainte d'infra imposée par ailleurs.

3. Interface utilisateur

Question à trancher : Streamlit, Gradio, React (ou équivalent), ou CLI seule ?

Options sur la table :

- **Streamlit / Gradio** — démo rapide, quasi zéro code web.
- **React** (ou équivalent) — interface produit dédiée, plus de travail.
- **CLI seule** — aucune interface graphique, question → réponse en terminal.

Ce qui doit décider : Dépend de qui utilise le système et à quelle fréquence. Une démo interne se contente de Streamlit/Gradio. Un produit orienté citoyen final demande une interface dédiée.

Si stage(s) solo, 8 semaines

CLI comme livrable engagé ; Streamlit/Gradio en option, uniquement pour la soutenance — pas un vrai produit d'interface.

Si produit d'équipe, long terme

React (ou équivalent) devient légitime si le produit doit être utilisé par des citoyens non techniques, hors contexte de démonstration.

4. Base vectorielle

Question à trancher : Chroma, Qdrant, FAISS, ou pgvector ?

Options sur la table :

- **Chroma** — embarquée, zéro serveur, persistance et filtrage par métadonnées inclus.
- **Qdrant** — fonctionne aussi en mode local sans serveur ; fusion RRF native ; API identique en local et en serveur.
- **FAISS** — bibliothèque bas niveau, rapide, mais sans persistance ni métadonnées intégrées (à coder soi-même).
- **pgvector** — extension PostgreSQL, pertinent si une base relationnelle existe déjà.

Ce qui doit décider : À ce volume de corpus (quelques milliers de chunks), la performance brute ne départage rien — toutes les options sont instantanées à cette échelle.

Si stage(s) solo, 8 semaines

Chroma : zéro ops, le plus rapide à mettre en œuvre pour une personne seule.

Si produit d'équipe, long terme

Qdrant est un choix défendable si l'équipe veut la fusion RRF native et anticipe un passage en mode serveur sans réécrire de code (même API en local et en serveur).

5. Cache des requêtes

Question à trancher : Redis, cache local (mémoire/disque), ou aucun cache ?

Options sur la table :

- **Redis** — serveur de cache partagé, nécessite une infra à opérer.
- **Cache local** (ex. diskcache, lru_cache) — aucun serveur, persistance simple.
- **Aucun cache** — chaque question rejoue tout le pipeline.

Ce qui doit décider : Le besoin est réel (répondre vite aux questions fréquentes, ex. « durée du congé de maternité »), mais l'infra nécessaire dépend du nombre d'instances du service qui doivent partager le même cache.

Si stage(s) solo, 8 semaines

Cache local, pas de serveur Redis à opérer — cohérent avec le choix de Chroma pour la même raison (zéro ops).

Si produit d'équipe, long terme

Redis devient légitime si plusieurs instances du service tournent en parallèle et doivent partager le même cache.

6. Persistance et historique utilisateur

Question à trancher : MySQL/PostgreSQL, JSONL/fichiers, ou SQLite ?

Options sur la table :

- **MySQL / PostgreSQL** — base relationnelle complète, accès concurrent.
- **JSONL / fichiers** — pas de base de données, simple, versionnable sous Git.
- **SQLite** — base embarquée sans serveur, entre les deux.

Ce qui doit décider : Dépend de l'existence ou non d'un vrai historique multi-utilisateur avec accès concurrent.

Si stage(s) solo, 8 semaines Pas de base relationnelle. JSONL suffit pour le corpus ; pas de mémoire conversationnelle (hors périmètre assumé, chaque question traitée isolément).	Si produit d'équipe, long terme Une base relationnelle devient légitime si le produit doit garder un historique par utilisateur avec accès concurrent réel.
--	---

7. Framework RAG

Question à trancher : LangChain / LlamaIndex, ou boucle codée à la main ?

Options sur la table :

- **LangChain / LlamaIndex** — standard du marché, mise en place rapide en équipe.
- **Aucun framework** — boucle RAG explicite (~150 lignes), chaque étape écrite et comprise.

Ce qui doit décider : La boucle RAG est courte. L'argument principal en faveur d'un framework est la vitesse de mise en place à plusieurs personnes, pas une nécessité technique du problème lui-même.

Si stage(s) solo, 8 semaines Hand-rolled — comprendre et pouvoir déboguer chaque étape, ce qui se défend directement en soutenance individuelle.	Si produit d'équipe, long terme LangChain devient plus défendable si plusieurs personnes doivent contribuer rapidement selon des conventions partagées, au prix d'une compréhension fine de chaque étape interne.
--	---

8. Architecture globale

Question à trancher : Microservices, ou service unique (monolithe) ?

Options sur la table :

- **Microservices** — composants séparés (front, backend métier, IA), déployables indépendamment.
- **Monolithe** — un seul processus, plus simple à raisonner et à déployer seul.

Ce qui doit décider : Les microservices se justifient par une mise à l'échelle indépendante des composants ou une répartition claire des responsabilités par équipe — pas par principe.

Si stage(s) solo, 8 semaines Monolithe simple, un seul processus Python. Les microservices ajoutent une complexité opérationnelle sans bénéfice réel à cette échelle.	Si produit d'équipe, long terme Défendables si plusieurs équipes possèdent des responsabilités distinctes (ex. gestion utilisateurs vs traitement IA) et doivent déployer indépendamment les unes des autres.
---	---

9. Conteneurisation

Question à trancher : Docker, ou environnement virtuel simple ?

Options sur la table :

- **Docker** — reproductibilité de déploiement sur plusieurs machines.
- **venv + requirements.txt** — plus léger, suffisant pour un déploiement unique documenté.

Ce qui doit décider : Docker apporte de la reproductibilité de déploiement — utile dès qu'il y a plus d'un environnement cible (dev, prod, plusieurs machines).

Si stage(s) solo, 8 semaines venv + requirements.txt documentés suffisent pour la reproductibilité exigée (clone vierge → build → run).	Si produit d'équipe, long terme Docker devient légitime pour déployer de façon identique sur plusieurs machines ou environnements gérés par différentes personnes.
---	--

Méthode de réunion suggérée : traiter le point 0 en premier et l'acter par écrit avant de poursuivre — chaque point suivant s'y réfère explicitement (« si scénario A » / « si scénario B »). Pour le point 1 (souveraineté), viser un accord de principe avant de discuter des points 2 à 9, qui en découlent tous au moins partiellement.

Document de préparation de réunion — mis à jour au 10 juillet 2026.